

An Approach to Improve Test Path Generation: Inclination towards Automated Model-based Software Design and Testing

Parampreet Kaur¹, Ashish Kr. Luhach²

^{1,2}Department of Computer Science and Engineering
Lovely Professional University, Jalandhar, India
¹paramnagpal16@gmail.com
²ashishluhach@acm.org

Abstract: Model based Testing in software engineering is gaining widespread importance due to faster and automatic generation of test suites for validating software systems. Generation of tests is done from the analysis and design phase artefacts of requirements and specifications. Due to rapid growth in the software industry there comes an urgency of developing a full fledged, fully-tested and defect free systems at earliest possibility. Testing using Models is a novel approach utilizing the key concepts of black-box testing. While substantial part of testing process relies on the appropriateness and completeness of the Model to be used, an approach for designing/modelling a system under test (SUT) with more accuracy is discussed. It can then be combined to the traditional methods of Model-based testing to fill the loopholes and gaps which emerge during design and testing. The approach describes the functionality of the system in a much better way. In this approach we can view the front end as well as structure of the system together. The approach will prove to be promising in improving testing and quality of test cases.

Keywords: Model based testing, System under Test (SUT), Labview, State charts, Finite state machine(FSM).

I. INTRODUCTION

In recent years, Model-Based testing (MBT) has unfolded into being an effective alternative for programming-based testing methods [27]. Model based testing is structured, comprehensive, accurate and measurable form of testing the software which is independent of the programming language and the platform used for building of any system [21]. Testing is one of the dominant approach for determining the quality of systems. It is widely accepted in industries and academia [15]. But, Manual testing is time consuming, labor-intensive and susceptible to errors [25]. The use of applications (apps) has considerably increased in the recent years due to emergence of variety of utilities to provide usage to its customers to save time and perform their tasks efficiently [2]. Therefore, the applications need to be tested in least possible time. This leads to providing full coverage and detection of faults at earlier stage in order to produce a qualitative software, which saves both effort and cost spent on testing them at later stages. The test cases generated, serve as a basis to develop and validate the customer needs to verify if the progress towards building the software is carried out in right direction or not well before the programming begins. The concept behind model-based testing is to generate test cases by systematically analyzing specification models [27]. Models can be designed using

standards such as UML, Markov chains, finite state machines, formal grammars[8]. A proficient Tester cannot afford to avoid Models. There are no such models which can meet all the objectives and goals of the system testing [25]. Decisions have to be made as to which model best suits the test generation process to get the desired outputs. Most of the studies reveal issues and problems [1] [2] [16] related to improper coverage of the models by the derived tests. A General overview of model-based testing is shown in the Fig 1.

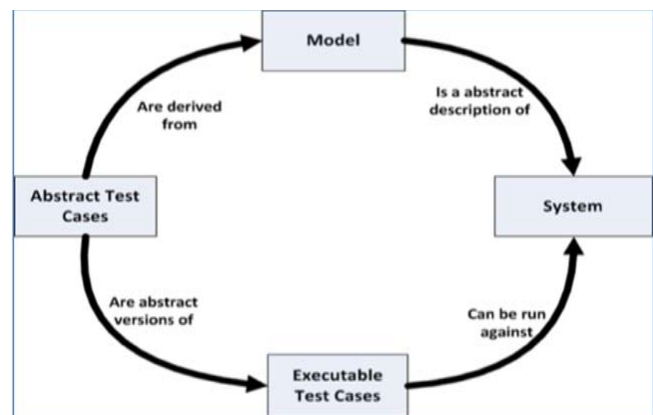


Fig. 1. Basic MBT process [10] showing extraction of test cases from system models.

Current software applications are often large and complex. Formal models have been explicitly used to automatically generate test cases [13]. But simply modelling them using formal methods such as UML, Markov chains, Petri-nets or finite state machines is not enough. Some more detailed modelling has to be done to uncover the defects and faults that might arise later [5].

The paper has been structured as follows. The next section discusses an overview of the related work taken up for modelling and testing. Based on this various advantages and limitations have been summarized at the end of section II. Section III describes the results of recent surveys conducted to have an insight about hold of Model-Based testing in industry and academics. Also it highlights the major application areas and testing levels which extensively utilize models for testing. Section IV describes the proposed approach to improve the

design of models and combining them to earlier Model testing approaches. Advantages are discussed after this section.

II. LITERATURE SURVEY

The research paper[2] describes a methodology of implementing MBT for testing and validating mobile phone applications. It discusses various advantages of using this technique in terms of automatic test case generation, error detection, scrutinized quality of tests, reduction in the cost and time due to evolution of models rather than program instructions. The authors also encountered certain challenges in terms of portability of test cases and requirement of expertise in the field of modelling the test cases using UML diagrams. Their study focused on Event Sequence Graph modelling technique to test Android applications. This paper in the beginning imposed a question if the concepts of MBT could be used to validate the functional requirements of mobile apps, which later provided experimental results that MBT combined with Event sequence graphs can be used as an effective method to model and generate test suites for mobile apps developed on this platform[2].

The study[16] proposed tools under AGEDIS consortium for automated model driven test generation and execution for distributed systems. These models contain a feedback loop for combining coverage and fault analysis tools with a test generator and execution framework [16]. The user has to input 3 types of information: a) The UML behavioral model of System under test, b) The test execution parameters c) test generation strategies to be implemented. The first two inputs are entered using UML modelling tool in combination with AGEDIS profile whereas third input is entered through XML editor[16] [17] [18].

A Research work[14] designed and generated test cases from UML state chart diagrams. In this Function Minimization Technique is applied and to generate test cases automatically from state chart machines. Firstly, a state chart diagram is constructed, and then it is traversed by depth first search algorithm to select predicates for initial data set. This technique achieves adequate test coverage without increasing the number of test cases. The research claims to achieve coverage in terms of State coverage, transition and transition pair coverage. The name given to the approach is AGeTeSC i.e. Automatically Generating Test cases from State Charts. It has been implemented using java and Net Beans IDE by taking Soft-drink Vending machine as a case study in a prototype tool named JUnit. Rational Rose tool is used to produce UML state diagrams and related artefacts for effective test generation[14].

In a paper presented by [20], The authors have used activity diagrams for the test generation paths in which a UML activity diagram is converted to XMI file and evaluated with cyclomatic complexity procedure. Then out of possible test cases generated, Optimization algorithm is used to select best test path. A Research study [9] discusses various algorithms to derive test cases from the graphs. Brute-Force, Greedy-set

coverage and prefix-graph coverage techniques are explained. Better algorithms are designed to minimize the cost of test paths. Detailed analysis of Prime-Path coverage is done. The main focus is on the test requirements. They have compared three solutions for minimum test path problem. Out of all the approaches, the prefix graph coverage solution seems preferable.

Another paper [19] analyzes and generates test suites from UML Interaction diagrams by using condition slicing approach. The approach is accomplished by construction of a UML interaction diagram. It is then converted to a Dependency Graph. The conditional predicates on Dependency graph are selected and computation of the slices is done corresponding to each conditional predicate. At the end Test cases are generated related to testing criteria specified. The approach is meant for the cluster level testing and thus no redundant test cases are generated. Since interaction diagrams are used so therefore interaction faults are detected during test phase.

A review paper[3] authored in 2015 discussed about Symbolic testing and model based testing (SMT) of software systems. It generalizes the idea of MBT tools and highlights the problems faced while MBT is in process. In this paper the author used a test generator based on symbolic approach. Symbolic approach is extended to incorporate changes and modifications required while executing MBT. The problem of exponential explosion of test cases was depicted in this paper. A technique of combining white box testing strategies with black box functional testing is presented to reduce the quantity of unlimited test cases. Then emerges the problem of unreachable states with MBT which can be resolved with integration of SMT technology. The paper also described the lack of coverage and traversal of all the states in case of large projects where only partial coverage is possible.

Model based testing in Practice[21] comprises of various advantages in testing arena. Most of the errors are due to inappropriate or incomplete models and programs. MBT ensures cost effectiveness to a large extent and extensive model coverage criteria. The study in this paper is confined to functional/operational testing. Several similarities and differences are specified between program verifiers and model based testing methods depicting the advantages and key points in this area. Model based methodology restricts the data types to be finite and small leading to extremely faster reasoning capability as compared to code based approaches. The support of model-based testing in industries will develop potential knowledge and skill in formal modelling and the operation of automated verification tools [21]. Work presented in [24] specifies various advantages and techniques which can be used for effectively carrying out MBT in industries and at academic levels for generating fruitful test cases automatically, thus, saving lots of effort and cost spent during traditional white box testing strategies.

In literature review, work related to transforming the design artefacts or the model designs into considerable test paths using automated designing and testing tools has been discussed. Most of the studies have shown and elaborated the usage of UML diagrams in their respective approaches to deal with the test case generation methods. Some have implemented finite state machines also. It is reported that errors revealed by executing the generated tests are mostly due to incorrect stipulations, requirements and codes. Many defects also arise due to inefficient modelling designs.

III. MBT SURVEY RESULTS

In many of the survey papers[6][7][11] [12] authors have presented motivation behind using MBT describing well documented results. Applications such as embedded systems domain, enterprise IT, Web Applications, Software infrastructure, Communications, gaming and education/ research are in the current users of MBT as shown in fig 2. They concluded in their surveys that the expectations regarding MBT are very high because the models help to manage complexity of a system. Test designing process starts earlier and models play a major role to improve the communication gap between the stakeholders and the developers. It leads to an interpretation that Model based testing proves to be largely effective and efficient for system testing i.e. validation of the requirements to check if they meet user needs or not. As compared to Unit testing, Integration testing and component testing, 80% results are obtained correct if MBT is used for conducting system level tests [1][7].

The results prove MBT to be an effective technology. Results report considerable improvement in the quality and saves time and cost as shown by fig 3.

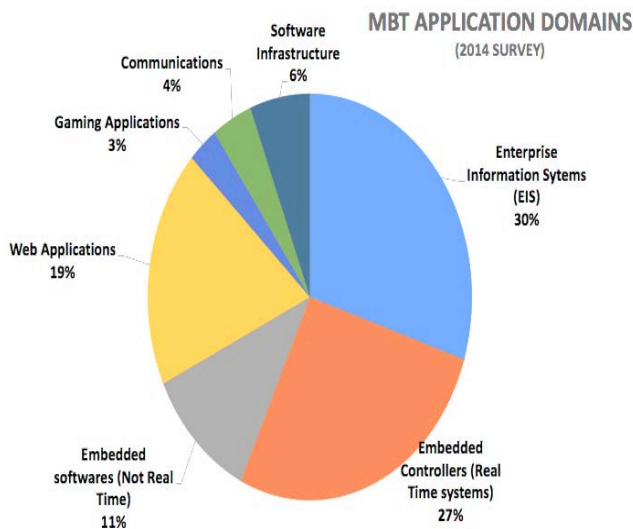


Fig. 2. Application fields of model based testing [1]

The average testing times is reduced by 25% [7]. Thus, organizations show a favorable view towards MBT. Possibly,

around 72% testers reported usage of model based testing in future for testing different applications.

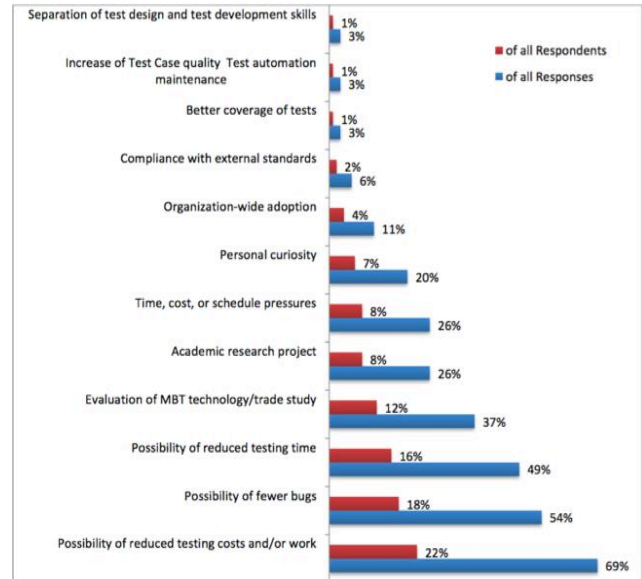


Fig. 3. Survey results about factors for using model based testing[6].

IV. PROPOSED APPROACH AND IMPLEMENTATION

In this section, a formal approach to design a software is explained. We have implemented the designing in LabView [28] [23] software version 2015. Two separate designs are built as per the user requirements. Then we follow the earlier strategies using one of the UML diagrams to further enhance the model. The need of designing any software using LabView assists in analyzing the system behavior in more detail. We divided the system into two parts: Front-end part and Logical part. In this paper, Modelling of only front-end part is discussed.

For keeping things simple to understand, design and testing of a calculator is taken up. A binary to decimal calculator is first of all designed using LabView [23] Software. There are two important panels available: a) Front panel b) Model Diagram panel. The system design is carried out in front panel. All the necessary inputs required are graphically designed in this panel. For representation of binary values, we have used four LEDs: LED A, LED B, LED C and LED D. If the user wants to input a binary number for example say 1100, then first two LEDs are set to ON state rest two are set to OFF state. In the fig shown below, we set LED A, B, D as ON, and LED C as OFF. This indicates that the input binary string is 1101 for the calculator shown below in the fig 4. The Output will be reflected in two different formats. Numeric Output and Analog indicators are used to depict the result of the computation. In fig 4, the decimal result of binary number 1101 is highlighted on right hand side in two different formats. When LED is result of the computation. In fig, the decimal result of binary

number 1101 is highlighted on right hand side in two different formats. When LED is turned green, it indicates a 1 or ON state. When LED is black, it indicates a 0 or OFF state.

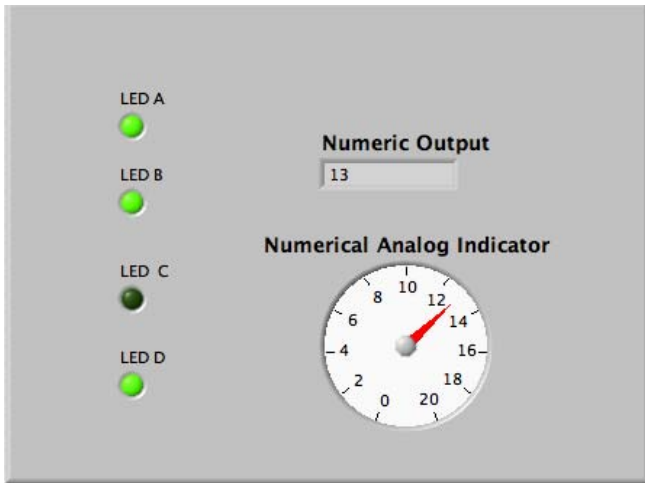


Fig. 4. Design of front panel of binary to decimal calculator

The corresponding model of the calculator shown above is depicted in the fig 5. In this model, the four LEDs used in front panel have been modelled to show the actual connections with rest of the components. Every LED is first of all connected to a Boolean to (0,1) function. To make things simpler to understand, every output is then fed into a multiplier along with a relevant constant value (8,4,2,1) to get the desired output on screen panel. Proper connections are made for the logical flow of data as shown in the fig 5 below. When results are displayed then we may abort the execution if needed. Moreover, we can pause it in between to view the intermediate results as shown in the fig 5 below. The output is then displayed in the numeric indicators of the front panel as shown in fig 5.

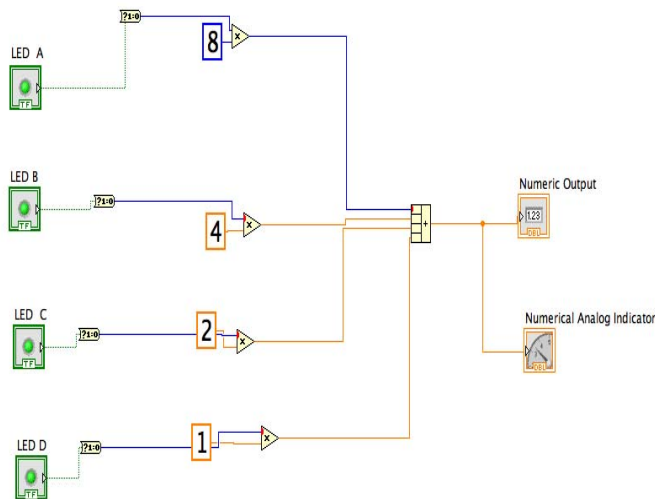


Fig. 5. Modelling diagram for binary to decimal calculator.

If you want to view the actual intermediary results of the execution, then “highlight execution” needs to be set ON. In the fig 6, since LEDs produce Boolean results, therefore a converter is applied to convert the input to 0 and 1. After that all the in-between produced results are added to produce final result. The 4-input adder is used which in this case is a compound arithmetic delivering one output.

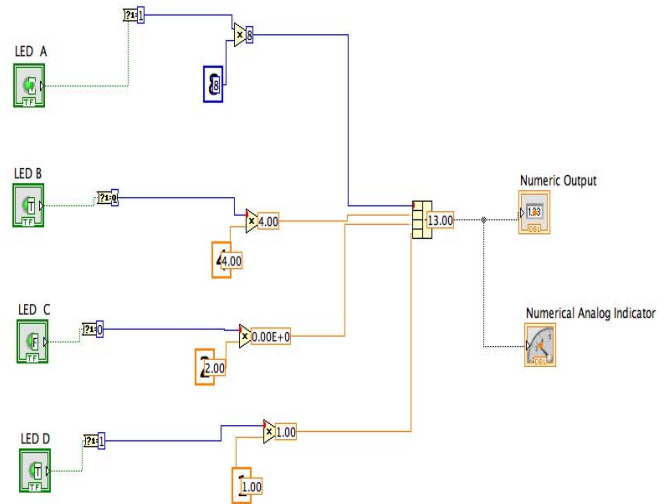


Fig. 6. Intermediate Model of the system showing intermediate results before actual output.

The diagram can be flattened to XML using Flatten to xml function and can serve as an important artefact for deriving tests using any of the techniques discussed in the literature survey. The XML can be converted into a State-chart machine diagram using SCXML- State Chart Extensible Markup Language. But in this case we have considered demonstration of sequence message diagram (SMD) for analyzing the behavioral aspects of the system. Plant UML is used to derive a sequence chart as shown in the fig 8. When the code is executed in Plant UML. It dynamically generates the SMD. The generated sequence chart is only partial depiction of message events occurring between different components.

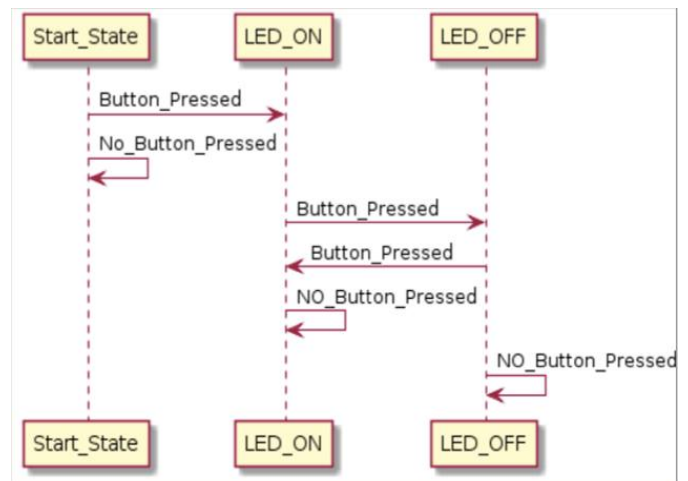


Fig. 7. Partial Sequence message diagram(SMD)

The code to generate a sequence diagram for the display unit of the application is mentioned below.

Start_State-> LED_ON : Button_Pressed

Start_State->Start_State :No_Button_Pressed

LED_ON -> LED_OFF: Button_Pressed

LED_OFF -> LED_ON: Button_Pressed

LED_ON -> LED_ON: NO_Button_Pressed

LED_OFF -> LED_OFF: NO_Button_Pressed

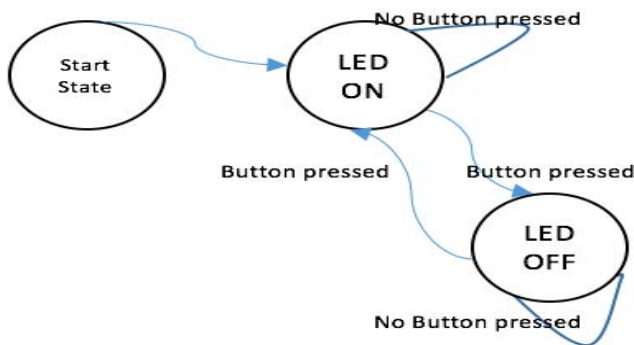


Fig. 8. FSM generated from the front panel design

The finite state machine (FSM) with three states: start state, Led On and Led Off State is shown in fig 8. The FSM is required to be built from the partial SMD so as to ease the test path derivation. For feeding the node and edge information in the tool, we require to change the nodes as 1,2,3 and so on. Let's assume Start state→1, LED_ON→2 and LED_OFF→3. This technique removes any self-loops from the FSM and reduces it to the minimized Directed graph (MDG) as shown in fig 9.

We have used and compared two test coverage methods [9] [15]: a) Edge-Pair Coverage b) Prime-Path Coverage. The results are shown in tables I and II. The Prime-Path coverage yielded lesser test paths thereby reducing the number of tests. Although same coverage is provided by both methods but the effort is considerably minimized when later technique is used [9]. The left portion of table I and II shows the number of test paths. The right portion presents the number of test requirements for traversing the complete graph. The test cases have been designed using Graph Coverage Web application [15].

TABLE I: 5-Test paths required to traverse the graph completely using edge pair technique

Test Paths	Test requirements coverage with edge-pair
------------	---

[1,2,3,2,3,3]	[1,2,3], [2,3,2], [2,3,3], [3,2,3]
[1,2,3,2,2,3,3]	[1,2,3], [2,3,2], [2,3,3], [3,2,2], [2,2,3]
[1,2,2,2,3,3]	[1,2,2], [2,3,3], [2,2,3], [2,2,2]
[1,2,3,3,2,3,3]	[1,2,3], [2,3,3], [3,2,3], [3,3,2]
[1,2,3,3,3]	[1,2,3], [2,3,3], [3,3,3]

TABLE 2: 2 -Test paths required if prime-path coverage technique is used.

Test Paths	Test Requirements that are toured by test paths directly
[1,2,3,2,3,3]	[1,2,3], [2,3,2], [3,2,3], [3,3]
[1,2,2,3,3]	[2,2], [3,3]

We have used both edge-pair as well as prime-path coverage criterions to find test paths. Although there are many other algorithms like Chinese postman for finding shortest paths [26], but this method has provided prompt results. The results in the Tables I and II interpret that by using this approach we get minimum number of test requirements and test paths if implementation is done using prime-path coverage.

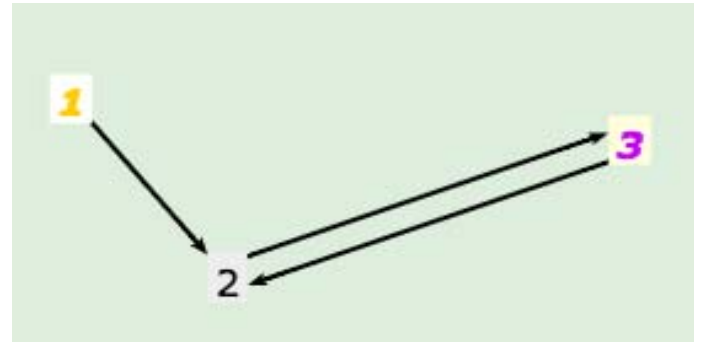


Fig. 9. Minimized Directed Graph (RDG) of system to derive test paths

For eg: The test path [1,2,3,2,3,3] indicates that if this path is followed by the system, then the requirements to be covered are [1,2,3], [2,3,2], [3,2,3], [3,3].

Advantages/Limitations

MBT is a Specification validation approach. Henceforth, Model based testing proves to be Cost-Effective Approach [4], helps Improve communication with stakeholders, detects more no. of errors than manual detection [22], aids easier Analysis of the failures, improves test quality and leads to easier update of test suite. It enlists certain limitations also such as Scalability problem, Outdated requirements problem and is mostly limited to functional testing [22].

V. CONCLUSION

MBT proves to be the effective and efficient approach towards generating test cases from UML models if applied on a particular set of requirements [13]. LabView graphical programming needs to be combined along with UML

modelling for better results. If the software design artefacts lack appropriate details about the functioning of the system's behavior, then obviously it may lead to ambiguous test cases as well. So in our approach we have tried to integrate the modelling of the systems or software controllers using LabView. This approach possibly may try to overcome the challenges and the issues faced while reusing the design models to produce tests. Since model-based testing is primarily dependent on the models designed in the design phase, therefore accurate details about the system's data flow needs to be considerably very well designed. The tester need not delve into the details of the programming language used for accomplishing this type of testing and therefore the errors, faults and defects can be detected during initial stages of system lifecycle, drastically saving the cost, efforts and time.

Keeping in view the shortcomings faced while executing MBT in industries by the professionals and other testing teams such as inability to cope with validating the implementation errors, MBT can be integrated with few white box testing methodologies such as consideration of cyclomatic complexity of graphs can be done. Also when coding is almost completed the code modules can also serve as input to the production of models if the project size has lesser LOC strength. The scalability issues can be resolved by constraining the generation of test cases to a few localized modules rather than on entire system altogether. Partial modules should be focused and errors should be detected to test one functionality at a time. This technique can considerably reduce the exponential explosion problem as depicted by [3]. The problem of unreachable states can be resolved if we partition a large system into smaller modules. Then derive intermediate graphs to execute full node and edge coverage. generate models and design statemachines using labview, the results can be more overwhelming. To cater to the problems of outdated requirements, automated toolset should be used where the probability of requirement modification and constant update of models can be easily handled. MBT can prove to be the better system testing technology than manual code based methods for embedded and communication systems [1].

REFERENCES

- [1] Robert V. Binder, B. L., Model-based Testing: Where Does It Stand? *Quality Assurance*, 13 (1), 1-9, 2015.
- [2] Farto., G.C. and Endo A.T. *Evaluating Model based Testing Approach in the context of Mobile Applications*. Electronic Notes in Theoretical Computer Science 314, 3–21, 2015.
- [3] O. O. Letychevskiy, *Paradigms Of Model-Based And Symbolic Testing Of Software Systems*, Cybernetics and Systems Analysis, Vol.51, No.5. 2015
- [4] Hetal, Thanki and SM Shinde, *Test case generation and minimization using UML activity diagram in model driven environment*. International Journal of Computer and Organization Trends. Vol.9, No.10, 2014
- [5] Raluka Marinescu, Cristina Secleanu., *A research overview of tool supported model based testing of requirements based designs*, Advances in computers, Vol 98., pp.89-140, 2015.
- [6] Weissleder S., Schlingloff H. *An evaluation of model-based testing in embedded applications*, Proceedings - IEEE 7th International Conference on Software Testing, Verification and Validation, ICST 2014, art. no. 6823884, pp. 223-232, 2014.
- [7] R. V. Binder, "Model-based Testing User Survey : Results and Analysis," 2012.
- [8] I. K. El-far and J. a Whittaker, "Model-Based Software Testing," *Encyclopedia of Software Engineering*, pp. 1–22, 2001.
- [9] N. Li, F. Li, and J. Offutt, "Better algorithms to minimize the cost of test paths," *Proceedings - IEEE 5th International Conference on Software Testing, Verification and Validation, ICST 2012*, pp. 280–289, 2012.
- [10] J. Naveen, "Model based testing," HCL White Paper, June, 2013.
- [11] M. Mussa, S. Ouchani, W. Al Sammane, and A. Hamou-Lhadj, "A survey of model-driven testing techniques," *Proceedings - International Conference on Quality Software*, pp. 167–172, 2009.
- [12] A. C. Dias Neto, R. Subramanyan, M. Vieira, and G. H. Travassos, "A survey on model-based testing approaches," *Proceedings of the 1st ACM international workshop on Empirical assessment of software engineering languages and technologies held in conjunction with the 22nd IEEE/ACM International Conference on Automated Software Engineering (ASE) 2007 - WEASEL Tech '07*, no. January, pp. 31–36, 2007.
- [13] A. Bertolino, E. Marchetti, and H. Muccini, "Introducing a Reasonably Complete and Coherent Approach for Model-based Testing," *Electronic Notes in Theoretical Computer Science*, vol. 116, no. SPEC.ISS., pp. 85–97, 2005.
- [14] Swain, R., Panthi, V. and Mohapatra, D. P., *Automatic Test Case Generation from UML State Chart Diagrams*. International Journal of Computer Applications, 42(7), pp.26 – 36, 2012.
- [15] P. Ammann and J. Offutt, *Introduction to Software Testing*. 2008.
- [16] Hartmann, J. and Nagin, K., The AGEDIS tools for Model Based Testing. *Proceedings of the ACM SIGSOFT international symposium on Software testing and analysis*, pp. 129-132, 2004.
- [17] AGEDIS Consortium, *Intermediate Language 2.0 with Test Directives Specification*, <http://www.agedis.de>
- [18] Craggs I., Sardis M., and Heuillard T., AGEDIS Case Studies: Model-based testing in industry. *Proc. 1st European Conference on Model Driven Software Engineering*, 106- 117. imbus AG December 2003.
- [19] Swain, R., Panthi, V. and Bahera, P.K. *Test Case Design using Slicing of UML interaction Diagram*. Proceedings of 2nd International Conference on Communication, Computing & Security [ICCCS-2012], Elsevier; 2012.
- [20] Kyaw, A.A. and Min M.M. *An Efficient Approach for Model Based Test Path Generation*. International Journal of Information and Education Technology, Vol. 5, No. 10, October 2015.
- [21] Mark Utting. *Position Paper: Model Based Testing*. University of Waikato, New Zealand, 2008.
- [22] Utting, M., Pretschner, A. and Legeard, B., *A taxonomy of model-based testing approaches*. Journal of Software: Testing, Verification and Reliability, 22: 297–312, 2012.
- [23] [https://en.wikipedia.org/wiki/State_machine_\(LabVIEW_programming\)](https://en.wikipedia.org/wiki/State_machine_(LabVIEW_programming))
- [24] Swain, S.K., Pani, S.K. and Mohapatra, D.P., *Model based object-oriented software testing*. Journal of Theoretical and Applied Information Technology 2010.
- [25] J. Dingel, W. Schulte, I. Ramos, S. Abrahão, and E. Insfran, *LNCS 8767 - Model-Driven Engineering Languages and Systems*. 2014.
- [26] Kaur, Parampreet., *Coverage Analysis and Chinese Postman Algorithm for efficient Model- approaches*. Journal of Software: Testing, Verification and Reliability, 22: 297–312, 2013.

- [27] M. Utting, B. Legeard, *Practical Model-Based Testing: A Tools Approach*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2007.
- [28] B. Hamed, "Design & Implementation of Smart House Control Using LabVIEW," *International Journal of Soft Computing and Engineering (IJSCE)*, vol. 1, no. 6, pp. 98–106, 2012.